

Troubleshooting & Debugging

Biostatistics in R and Python — a field guide

Most of the time you lose to a script is not spent writing it — it is spent finding out why it does not work, or worse, why it runs cleanly but gives the wrong answer. This guide collects the errors and silent traps that come up again and again in biological data analysis, organised by where they bite, with the fix shown in both R and Python. It is meant to be skimmed when you are stuck: find your symptom, read the cause, apply the fix. The hardest bugs in biostatistics are the silent ones — code that produces a number that is simply wrong — so the statistical-pitfalls section is the most important one here.

1. The Debugging Mindset

Before any specific fix, four habits solve most problems. **Read the actual error** — the message almost always names the problem and the line. **Reproduce it minimally** — shrink to the smallest code and data that still fails; the bug usually reveals itself in the shrinking. **Inspect your data, do not assume it** — look at types, dimensions, and missing values before computing. And **change one thing at a time** — a controlled change tells you what mattered; ten at once tell you nothing.

In Python, read the traceback from the bottom up. Python prints the chain of calls with the actual error on the LAST line — start there for the error type and message, then read upward to find the line in your code. R prints the error first, then `traceback()` shows the call stack. In both, the goal is the same: which line, and what was wrong with the value it saw.

2. Inspect Before You Compute

The single most useful debugging move is to **look at the object** — its type, shape, and a few values. Half of all analysis bugs are a variable that is not what you think it is (a string where you expected a number, a data frame with 0 rows, a column full of NA). These commands cost nothing:

Check	R	Python
Type of an object	<code>class(x); typeof(x)</code>	<code>type(x)</code>
Structure / dtypes	<code>str(df)</code>	<code>df.info(); df.dtypes</code>
Dimensions	<code>dim(df); length(x)</code>	<code>df.shape; len(x)</code>
First few values	<code>head(df); df[1:5,]</code>	<code>df.head()</code>
Any missing?	<code>sum(is.na(x))</code>	<code>x.isna().sum()</code>
Range / summary	<code>summary(x); range(x)</code>	<code>x.describe()</code>
Unique values	<code>unique(x); table(x)</code>	<code>x.unique(); x.value_counts()</code>

3. Loading Data: The First Place It Breaks

Symptom Numbers behave like text; sorting puts "10" before "2".

→ **Cause** A column was read as character/string — often because of a stray symbol, a thousands separator, or a decimal comma.

→ **Fix** Read with the right options, then convert explicitly.

```
R
df <- read.csv('data.csv', na.strings = c('NA', '', 'NaN', '-'))
df$x <- as.numeric(df$x) # force a column numeric (check NAs after)
# European decimals: read.csv2(...) uses ',' as the decimal point
```

```
Python
df = pd.read_csv('data.csv', na_values=['NA', '', 'NaN', '-'])
df['x'] = pd.to_numeric(df['x'], errors='coerce') # bad values -> NaN
# European decimals: pd.read_csv(..., decimal=',')
```

The Excel gene-name disaster. Excel silently converts gene symbols like SEPT2, MARCH1, and DEC1 into dates, and long IDs into scientific notation. If your gene names look like "2-Sep" or "1.00E+09", that already happened. Import raw text files (CSV/TSV) directly into R or Python, keep identifiers as character/string, and never round-trip gene lists through Excel.

Symptom Wrong number of columns, or the header is in the data.

→ **Cause** Wrong delimiter (comma vs tab), comment lines at the top, or no header.

→ **Fix** Set sep/delimiter, skip comment lines, and say whether a header exists.

```
R
read.delim('counts.tsv', comment.char = '#')      # tab-separated, skip #
read.csv('d.csv', header = FALSE, skip = 2)
```

```
Python
pd.read_csv('counts.tsv', sep='\t', comment='#')
pd.read_csv('d.csv', header=None, skiprows=2)
```

4. Missing Values That Poison a Result

Symptom mean(), sd(), or cor() returns NA / NaN for no obvious reason.

→ **Cause** A single missing value propagates through the whole calculation by default.

→ **Fix** Tell the function to ignore missing values — but first understand WHY they are missing.

```
R
mean(x)                #> NA    (one NA poisons it)
mean(x, na.rm = TRUE)  #> 7.4   (drop NAs)
cor(x, y, use = 'complete.obs') # pairwise-complete for correlation
df[complete.cases(df), ]      # keep only fully-observed rows
```

```
Python
x.mean()                # pandas skips NaN by default
np.mean(a)              #> nan   (numpy does NOT skip)
np.nanmean(a)           #> 7.4   (nan-aware version)
df.dropna(subset=['x', 'y']) # drop rows missing x or y
```

Dropping NAs is a statistical decision, not a formatting one. Discarding incomplete rows can bias your result if the data are not missing at random (e.g. low-expressed genes are missing more often). Report how many rows you dropped and why. In R remember NA (missing) is distinct from NaN (not-a-number) and NULL (absent); in Python None and np.nan behave differently in comparisons (np.nan == np.nan is False).

5. Indexing, Subsetting & Off-by-One

Symptom Results are shifted by one, or you grabbed the wrong column.

→ **Cause** R counts from 1 and includes both ends of a range; Python counts from 0 and excludes the end.

→ **Fix** Keep the difference in mind, and index by NAME rather than position when you can.

```
R
x[1]          # FIRST element (1-based)
x[1:3]        # elements 1,2,3 (end included)
df[, 'gene']   # by column name, robust to reordering
```

```
Python
x[0]          # first element (0-based)
x[0:3]        # elements 0,1,2 (end EXCLUDED)
df['gene']    # by column name
```

Symptom pandas warns 'A value is trying to be set on a copy of a slice' (SettingWithCopyWarning).

→ **Cause** You filtered a DataFrame and then assigned to the result, which may be a view, not a copy.

→ **Fix** Make an explicit copy, or assign with .loc in one step.

```
Python
sub = df[df.tissue == 'leaf'].copy() # explicit copy, then edit safely
sub['z'] = sub['x'] * 2
# or in one go: df.loc[df.tissue=='leaf', 'z'] = ...
```

6. Statistical Pitfalls in Biostatistics

These are the dangerous bugs: the script runs and prints a number, but the number is wrong. None of them throws an error — only knowing the statistics catches them.

Wrong orientation of the data matrix. Expression tools disagree about whether samples are rows or columns. Feeding genes-as-rows to a function expecting samples-as-rows transposes your entire analysis silently. Always check dimensions against your sample count first.

```
R
dim(counts)    #> 20000  12  (genes x samples?)  confirm before DESeq
```

```
Python
X.shape        #> (12, 20000)  scikit-learn wants samples x features
```

Skipping the test's assumptions. A t-test assumes roughly normal data and (by default) equal variances; ANOVA the same. On skewed counts or tiny samples these can mislead. Check first, or use a test that does not assume normality.

```
R
shapiro.test(x)          # normality check
t.test(x ~ g)            # Welch by default (unequal var) – good
wilcox.test(x ~ g)       # nonparametric fallback
```

```
Python
scipy.stats.shapiro(x)
scipy.stats.ttest_ind(a, b, equal_var=False)  # Welch – set this!
scipy.stats.mannwhitneyu(a, b)               # nonparametric
```

scipy's default t-test assumes equal variance. `scipy.stats.ttest_ind` uses `equal_var=True` by default, the opposite of R's `t.test` (which defaults to Welch). Unless you have checked that variances are equal, pass `equal_var=False` so the two languages agree and you do not overstate significance.

Forgetting to correct for multiple testing. Testing 20,000 genes at $p < 0.05$ yields ~1,000 false positives by chance. Adjust the p-values (Benjamini–Hochberg / FDR) and threshold on the adjusted value — this was central to Chapters 13 and 16.

```
R
padj <- p.adjust(pvals, method = 'BH'); sum(padj < 0.05)
```

```
Python
from statsmodels.stats.multitest import multipletests
padj = multipletests(pvals, method='fdr_bh')[1]
```

The wrong baseline in a model. In `lm/glm` the factor's reference level sets what coefficients are compared against. R picks the first level alphabetically — so "control" must be made the reference, or your effect sizes flip sign and meaning.

```
R
df$group <- relevel(factor(df$group), ref = 'control')
```

```
Python
smf.ols("y ~ C(group, Treatment(reference='control'))", df).fit()
```

log(0), tiny p-values, and the log scale. Log-transforming counts hits $\log_2(0) = -\text{Inf}$; add a pseudocount. Very small p-values underflow to exactly 0 in double precision, so compare and store them on the log scale. And never average values that are already on a log scale and report it as a linear mean.

```
R
logCPM <- log2(cpm + 1)          # pseudocount avoids -Inf
# keep tiny p-values usable: -log10(p) instead of p
```

```
Python
log_cpm = np.log2(cpm + 1)
neglogp = -np.log10(pvals)       # for volcano plots & storage
```

Degenerate inputs. Correlation or a t-test on a constant vector (zero variance), a group with $n = 1$, or all-identical values returns NA, NaN, or a meaningless result rather than erroring. Filter out zero-variance features (e.g. genes with no reads) before testing.

7. Numbers, Floating Point & Infinity

Symptom `0.1 + 0.2 == 0.3` is FALSE; an equality check on decimals fails.
→ **Cause** Floating-point numbers are approximate; `exact ==` rarely holds.
→ **Fix** Compare with a tolerance, not equality.

```
R
isTRUE(all.equal(0.1 + 0.2, 0.3))    #> TRUE
```

```
Python
import math; math.isclose(0.1 + 0.2, 0.3)    #> True
```

Watch too for Inf / NaN creeping in from division by zero or `log(0)`, and for integer overflow on huge counts (use numeric/float). Check with `is.finite()` / `np.isfinite()` before a result flows into the next step.

8. Packages, Versions & Function Masking

Symptom A function suddenly behaves differently or 'could not find function'.
→ **Cause** Two loaded packages define the same name (masking), or a package isn't installed/loaded.
→ **Fix** Call the function with its package prefix, and mind load order.

```
R
dplyr::select(df, x)          # be explicit when dplyr & MASS both have select()
# install Bioconductor packages with BiocManager, NOT install.packages:
BiocManager::install('DESeq2')
```

```
Python
import pandas as pd; print(pd.__version__)    # check the version
# isolate project deps in a venv/conda env to avoid version clashes
```

"It works on my machine." Different package versions are a top cause of irreproducible results. Record them (`sessionInfo()` in R; `pip freeze` / `conda env export` in Python) and, for anything important, pin them — exactly the reproducibility habit from Chapter 19.

9. Plots That Don't Appear or Mislead

Symptom No plot shows up when running a script (it works in the console).
→ **Cause** ggplot/matplotlib objects must be explicitly printed or shown in scripts and loops.
→ **Fix** Print the plot object (R) or call `show/savefig` (Python).

```
R
p <- ggplot(df, aes(x, y)) + geom_point()
print(p)                      # inside scripts & for-loops you must print()
ggsave('fig.png', p)
```

```
Python
plt.scatter(df.x, df.y)
plt.savefig('fig.png', dpi=150)    # or plt.show() in interactive use
plt.close()                       # free memory in loops
```

Misleading plots are subtler: a **factor in the wrong order** (set the level order deliberately), points hidden by **overplotting** (use transparency or jitter), and rows silently dropped because of NA. If a plot looks wrong, plot the underlying numbers directly to check.

10. "It Worked Yesterday": Reproducibility Bugs

If results change between runs, suspect **randomness without a seed** (set one — Chapter 20), or **hidden state** in an interactive session where a variable lingers from deleted code. The cure is to

restart and run top-to-bottom from a clean session before trusting any result (Chapter 19). If paths break on another machine, you used an absolute path; switch to project-relative paths.

```
R
set.seed(42)          # before any random step; then Session > Restart R
```

```
Python
rng = np.random.default_rng(42)  # then Kernel > Restart & Run All
```

11. When Code Is Too Slow or Runs Out of Memory

On genomic-scale data, the usual culprit is a **growing loop**. Vectorise — operate on whole columns at once — instead of looping row by row, and pre-allocate rather than appending. If the data will not fit in memory, read it in chunks, keep only the columns you need, or use an out-of-core tool (data.table/arrow in R, Dask/Polars/chunked pandas in Python). Profile before optimising: measure which line is actually slow rather than guessing.

```
R
means <- rowMeans(mat)          # vectorised – not a for-loop over rows
system.time({ ... })           # time a block
```

```
Python
means = X.mean(axis=1)          # vectorised with numpy/pandas
%timeit func()                  # time it (Jupyter)
```

12. Debugging Tools at a Glance

To...	R	Python
Print a value mid-script	print(x); cat(x, '\n')	print(x)
See the call stack	traceback()	(read the traceback)
Pause & inspect (breakpoint)	browser()	breakpoint() # or pdb
Step into a function	debug(f)	import pdb; pdb.set_trace()
Turn warnings into errors	options(warn = 2)	warnings.filterwarnings('error')
Assert an expectation	stopifnot(n > 0)	assert n > 0, 'msg'
Inspect an object	str(x)	type(x); vars(x)

A Pre-Flight Checklist

Before you trust a result, run through this list:

- Did you LOOK at the data — types, dimensions, head, and missing-value counts?
- Are numeric columns actually numeric (not character/factor)?
- Is the data matrix the right way round (samples vs features)?
- Are missing values handled deliberately, and did you note how many you dropped?
- For any test: are its assumptions met, and is it the right test (paired? variances?)?
- Did you correct p-values for multiple testing and threshold on the adjusted value?
- Is the model's reference/baseline level the one you intend?
- Did you guard against log(0), zero-variance inputs, and tiny-p underflow?
- Did you set a seed, and does the script run clean top-to-bottom from a fresh session?

The golden rule of silent bugs. When a result looks surprising, your first hypothesis should be a bug, not a discovery. Re-check the data and the code before you re-check the biology. The most dangerous output is the one that looks plausible and is quietly wrong.